LLMChain 白皮书

第一部分 · 概览

人类交易靠发票，Agent 交易靠什么?

人类之间的每一笔交易都有凭证：发票、合同、银行流水。凭证让陌生人之间可以合作，也让纠纷有据可查。

AI Agent 正在成为新的经济主体。它们一天发起成千上万次推理调用，向不同的模型服务商付费，再把结果卖给下一个 Agent。但今天，**一次推理调用发生之后，没有任何凭证能证明它真的发生过、用的是声明的那个模型、价格是双方约定的那个价格**。

LLMChain 为 Agent 经济提供这张凭证，LLMChain 是一条把推理调用变成可验证、可结算、可组合的链上原语的独立公链。

基金会做链，社区做生态。基金会只负责一个足够薄、足够可信的验证与结算底座：provider 验证、VSR 收据、支付通道、质押与客观罚没。多方分账等会随 Agent 商业模式快速变化的逻辑，由社区在合约层开发、迭代，不进入链的内核。

LLMChain 三个要点

1. **可验证收据（VSR）是唯一上链的对象** —— 每一次被完成的推理调用生成一张收据，收据即结算。
2. **毫秒级调用，不等区块确认** —— 选服务商、付款、执行全部在链下完成，只有结算上链。
3. **造假可被抓、抓到即罚没** —— 隐蔽挑战 + 保真证明 + 质押罚没，让以次充好在经济上不划算。

关键参数

参数	数值
调用延迟开销	链下完成，不等区块确认

出块时间 / 最终性	【待定】约 1 秒 / 亚秒级确定性最终性
共识	纯质押 PoS，服务量不进入共识权重
服务商准入	普通 GPU 即可；GPU TEE 为可选机密级

一次调用的四步

1. **选择** —— Agent 查询链上挂单簿，按模型、价格、证明等级选定服务商
2. **付款** —— 通过 x402 建立支付通道，服务商链下当场决定是否接单
3. **执行** —— 服务商完成推理，返回结果与签名
4. **结算** —— 签发 VSR 收据上链，资金按收据结算

为什么是现在，为什么是我们

时机：Agent 正在成为推理的主要消费者

推理需求的主角正在从"人在聊天框里提问"转向"Agent 以及 Agent 之间自动调用"。Agent 调用的特点是：频次高、单价低、链路长（一个任务递归调用多个下游 Agent 与模型）、全程无人值守。

无人值守意味着**没有人会逐条核对结果**。这让信任问题从"体验问题"变成了"结算问题"。

问题：今天的推理市场无法自证

问题	具体表现
以次充好	中转商挂旗舰模型、报三折价，背地里换成便宜的开源小模型，用户无法逐字核对
调用无凭证	账单由平台单方面出具，Agent 之间转售推理时没有可追溯的结算事实
平台托管资金	聚合平台先收钱再分发，调用方必须信任中介

递归分账困难	Agent A 调 B、B 调 C，收入如何按链路自动分配，没有通用原语

现有项目大致分成两个阵营。**证明派**（如 OpenGradient、Phala、Gensyn）致力于证明"推理是否正确"，但没有结算层：证明交付后，谁该被付钱、谁该被罚没，不归它们管。**市场派**（如 Akash、io.net）已经有真实的算力与推理成交量，但对服务是否按声明发生不做任何证明。

LLMChain 站在两者之间：把可验证的服务事实与原生结算放进同一个状态机。

我们与现有方案的区别

	LLMChain	推理证明预言机（如 OpenGradient）	推理 L1（如 Flop）	去中心化 AI 网络（如 Bittensor）	中心化聚合平台
结算对象	单次调用 VSR 收据	供合约查询的证明结果	会话证明哈希	子网评分与排放	平台账单
资金托管	非托管，调用方直连服务商	—	链上	链上排放	平台托管
共识权重	纯质押，与服务量脱钩	—	质押 + 有用推理	验证者评分影响矿工收益	—
防以次充好	保真证明 + 隐蔽挑战 + 罚没	zkML / 乐观证明	指纹 + 抽样重执行	主观评分	依赖平台信誉

我们的核心判断：推理市场需要的不是"证明回答是正确的"（这在开放式生成任务中不可能做到），而是"证明调用是真实的"。把问题收窄，才能把证明做实。

我们是 x402 和 ERC-8004 缺的下半段。x402 解决了"单次调用怎么付钱"，ERC-8004 解决了"Agent 是谁"，两者都明确不处理"这次调用是否按声明发生、多方之间如何结清"。LLMChain 补的正是这一段：任何 ERC-8004 注册的 Agent、任何 x402 付款，都可以直接接入 LLMChain 的收据与结算，无需经过特定应用。AgentStore 是第一个客户，不是唯一入口。

代币与路线图

代币用途（代币名【待定】，下称 $LLM）

用途	说明
调用支付	Agent 用 $LLM（或经兑换的稳定币）支付推理费用
服务商质押	挂单前必须质押，造假即罚没
审计员质押	获得发起隐蔽挑战的资格，抓到造假获得赏金
验证人质押	参与 PoS 共识出块，获取出块奖励与协议费分成
治理	协议参数调整、模块升级投票

价值闭环

```
Agent 调用付费 → 协议费累积 → 质押收益提升 → 更多服务商与验证人质押
      ↑                                                    ↓
  更多 Agent 接入 ←  更低价格、更可信的供给  ←  供给规模扩大
```

调用量越大，协议费越多；协议费越多，质押需求越强；质押越多，造假成本越高、网络越可信。

路线图

阶段	时间	里程碑
测试网	【待定】	五大原生模块上线；开源模型自托管服务商接入
主网 Beta	【待定】	VSR 结算、审计员挑战与罚没生效；首批路由服务

主网	【待定】	闭源模型中转（保真证明）、GPU TEE 机密级服务
生态扩展	【待定】	多方分账合约标准、Agent 应用商店

第二部分 · 机制

为什么必须是一条独立公链

我们曾考虑把 LLMChain 做成部署在现有 EVM 链或 Rollup 上的合约集。最终选择独立 L1，原因有三。

理由一：收据是协议级原语，不是应用

VSR 收据是唯一穿透链下与链上的对象。每次调用的收据都要被签发、聚合、验证和结算，且调用频次远高于普通 DeFi 交易。在通用链上用合约实现，每张收据都要支付通用 EVM 的验签与存储开销；在独立链上，收据的签发、聚合、验证由**原生模块和预编译**直接支持，成本和吞吐都不在一个量级。

理由二：共识安全模型必须为推理市场定制

我们坚持一条第一原则：**服务量不得进入共识权重**。

LLMChain 采用纯质押共识：共识权重只来自质押；服务收益只由用户付费决定；罚没只由**客观失败**触发（如拿不出转发证据、签名不符），不由任何人的主观评分触发。这套规则需要在共识层定义，无法在别人的链上实现。

理由三：链下调用与链上结算的边界需要原生定义

调用必须是毫秒级的，不能等区块确认。这要求链原生理解"支付通道 + 链下接单 + 链上结算"这一模式：挂单簿可被高频查询、通道可被快速开设、只有完成的调用才上链。被拒绝的请求不消耗链上资源。这是一套完整的协议设计，不是几个合约能拼出来的。

我们刻意不做的事

- **不做协议级智能路由**：一次请求只发给一个服务商，一次会话只锁定一个服务商。用单点故障风险换取协议逻辑的可验证性；冗余与重试交给协议之上的路由服务。

- **不在链内核做多方分账**：Agent 商业模式变化太快，分账规则留在合约层，由社区演进。
- **不强制 GPU TEE**：硬件门槛会压缩供给；机密性与完整性是两种不同的保证，分开提供。

信任机制：如何让造假不划算

四层保障

层级	机制	保证什么
准入	服务商注册时验证质押与远程证明（attestation）	只有有身份、有抵押的服务商能挂单
完整性	Tier 0/1/2 分级保真证明 + 强制确定性复算	调用真的发生在声明的模型上
发现	审计员隐蔽挑战	服务商无法分辨真实调用与审计探测
惩罚	客观违约即罚没，部分罚没所得作为审计员赏金	造假的期望收益为负

保真证明：Tier 0/1/2 分级验证

不同调用对信任的要求不同，强制所有调用使用最高等级只会压缩供给。LLMChain 把保真证明分为三级，provider 在挂单时声明自己支持的等级，调用方按需选择；等级写入 VSR 收据的 proofTier 字段，结算与罚没规则按等级执行。

等级	机制	适用场景
Tier 0	双方签名收据 + 质押担保；依靠审计员隐蔽挑战事后发现违约	低单价、高频调用；provider 起步接入
Tier 1	Tier 0 + 抽样指纹 + 强制确定性复算：审计员抽样重放，输出必须逐位一致	开源权重自托管的默认等级
Tier 2	Tier 1 + TEE 远程证明：执行或上游转发在 enclave 内完成，并叠加随机强制 ZK 抽查	闭源模型中转（必须）、高价值调用

开源权重自托管是网络的核心供给：权重哈希公开，声明可被核对，从 Tier 0/1 起步即可接入。闭源模型中转是可插拔的许可制附加模块，必须达到 Tier 2：向上游的真实转发必须在 TEE 内完成，enclave 记录 TLS 会话证据；审计员随时可以发起隐蔽挑战，服务商拿不出转发证据即触发客观违约罚没。

借鉴前沿研究的三项升级

我们早期的设计使用 top-k logprob 分布加统计容差来识别模型指纹偏差。这一做法落后于已发表的研究：容差阈值缺少可靠的定量校准方法，设松了放过作弊，设紧了误罚老实 provider。我们直接借鉴以下三项成果，不从头重新推导：

- **强制确定性执行**（借鉴 Gensyn RepOps 思路）：约束算子实现与执行顺序，使同一输入在同一模型上产生逐位一致的输出。判定从"偏差是否可接受"变成"结果是否一致"，客观、二值、可复现，符合"验证人只做客观裁决"的原则。统计指纹保留为发现可疑行为的辅助信号，不作为罚没依据。
- **二分争议博弈**（借鉴乐观 TEE-Rollup 研究）：作为申诉机制。挑战方与 provider 对执行轨迹逐轮二分，把争议收敛到单个计算步骤，链上只需验证这一步。这样既降低单次裁决的误判成本，也让复杂案件能被拆小到链上可负担的验证粒度。
- **随机强制 ZK 抽查**：作为 TEE 被攻破后的第二道防线，详见下文。

ZK 在 LLMChain 中的角色

对大模型的每一次推理都生成零知识证明（zkML），目前在成本和延迟上都不可行。因此 LLMChain 不把 ZK 作为默认路径，而是放在三个"少量但关键"的位置：

- **随机强制抽查**：协议按随机数选中少量 Tier 2 调用，要求 provider 在规定时限内补交 ZK 证明。provider 无法预判哪一次会被抽中，因此"攻破一个 enclave 就能长期稳定作弊"的假设不再成立。ZK 的价值不在覆盖率，而在于让作弊的期望收益为负。
- **争议的最终一步**：二分争议博弈收敛到单个计算步骤后，该步骤可以用 ZK 证明在链上验证，取代重新执行。
- **可选的最高保证等级**：对小模型或高价值调用，调用方可以要求每次附带 ZK 证明，由 proof 模块的验证预编译在链上校验，费用由调用方承担。

ZK 证明系统与电路选型【待定】，将随 zkML 证明成本的下降逐步扩大适用范围；协议只固定"证明的验证接口"，不绑定某一种证明系统。

多厂商 TEE 信任根

Tier 2 目前依赖单一厂商的 TEE。路线图上，我们将在 SGX 之外至少支持一种机密计算信任根（AMD SEV-SNP 或机密 GPU attestation）。真正的难点在于跨厂商失效时的一致裁决：某一厂商爆出漏洞、大规模撤销证书时，链如何在不停机、不误罚老实 provider 的前提下降级运行。这一层我们尚无完整答案，不会先于实现宣称已解决；它排在递归结算旗舰演示之后，不阻塞主线。

审计员与验证人分离

角色	职责	质押对应
审计员	发起隐蔽挑战（找问题），不裁决	挑战资格
验证人	作为共识的一部分客观裁决、执行罚没与结算	出块权重

分离的原因很简单：如果审计员能自己裁定输赢，就有动机诬告或纵容。裁决权收归客观的链上逻辑，审计员只负责找问题、领赏金。

机密性是可选的独立服务

保真证明保证的是**执行完整性**，不是 prompt 的机密性。需要机密性的调用可以选择机密级服务商，这类服务商必须提供 CPU/GPU 联合远程证明，机密请求只路由到具备该能力的节点。普通级调用的完整性由保真证明、隐蔽挑战、质押罚没和客户端脱敏共同保障。

主要攻击与防护

攻击	防护
服务商以小模型冒充大模型	确定性复算 + 二分争议 + 罚没
中转商伪造上游调用	TEE 内 TLS 会话证据 + 随机强制 ZK 抽查，拿不出即罚没
服务商识别并特殊对待审计请求	隐蔽挑战与真实调用不可区分
审计员诬告	审计员无裁决权，挑战失败扣除挑战押金

验证人串通操纵服务商收益	服务收益只由用户付费决定，验证人无评分权
调用方完成调用后拒付	支付通道预锁资金，VSR 签发即结算

第三部分 · 构建

架构



五个原生模块

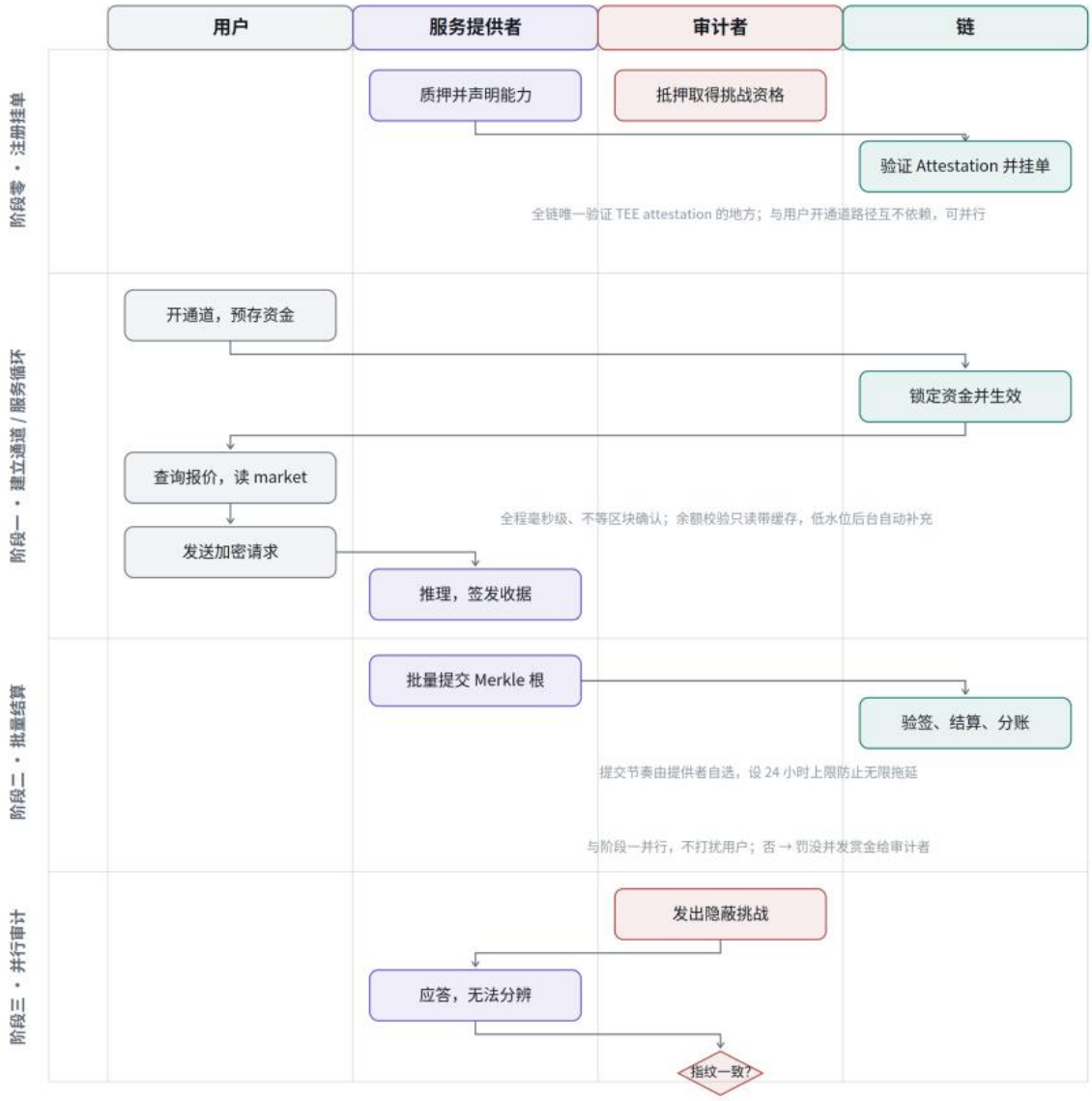
模块	职责
provider	服务商注册、质押、远程证明校验、退出与罚没
market	链上可查询挂单簿：模型、单价、证明等级、可用容量、可选机密级声明
channel	调用方与服务商之间的支付通道：开设、预锁、关闭
receipt	VSR 收据的签发验证、聚合与结算
proof	保真证明、隐蔽挑战、指纹判定与罚没执行

关于 market 挂单簿

market 不是 mempool 那种打包后即消失的交易缓冲区，而是一个持久的服务目录：服务商声明的挂单会一直有效，直到自己更新或撤下。

- **挂单由服务商自己声明**，验证人只在注册阶段把关一次（远程证明与质押）；
- 报价与容量的真实性交给市场纠偏：用户用脚投票、复购率、历史信誉；
- 容量字段是"上次同步的参考值"，允许滞后；真正的接单裁决由服务商在请求到达时链下做出。

一次调用的完整生命周期



关键设计

- **毫秒级，不等区块**：第 1–4 步全部在链下完成。调用延迟只取决于推理本身。
- **只有完成的调用才上链**：服务商拒绝的请求不产生收据，不消耗链上资源，不产生费用。
- **失败处理**：服务商拒单或超时，调用方直接换下一个候选；该逻辑可由路由服务自动完成。
- **收据可聚合**：高频调用的收据可在通道关闭时批量聚合提交，由预编译验证。

VSR 收据核心字段

字段	含义
callId	本次调用唯一标识
parentCallId	父调用标识（递归调用时）
rootCallId	调用链根标识
caller / provider	调用方与服务商地址
modelHash	声明的模型权重哈希
proofTier	证明等级（Tier 0/1/2）
inputDigest / outputDigest	输入输出摘要（不含明文）
amount	结算金额
authToken	父调用方签署的授权令牌（递归调用时）
signatures	双方签名

为什么用调用标识而非父收据关联？ 子调用发生时，父调用尚未完成、父收据尚未签发。用 `callId / parentCallId / rootCallId` 加授权令牌，receipt 模块即可在子调用结算时验证授权关系，多方分账规则再交给合约层处理。

生态角色与收益

角色	准入	质押	收入	罚没条件

服务商（Provider）	注册 + 远程证明	必需	调用费	客观违约：转发证据缺失、签名不符等
审计员（Auditor）	质押即可	必需（挑战资格）	罚没赏金	挑战失败扣除挑战押金
验证人（Validator）	PoS 质押	必需（出块权重）	出块奖励 + 协议费分成	双签、离线等共识违约
路由服务（Router）	无需许可	无	订阅费、API 接入费、服务商引流费	—
应用开发者	无需许可	无	自定义：分账合约、Agent 服务费	—
委托人	持币即可	委托给验证人或服务商	按比例分享收益	随被委托方罚没

路由服务：协议之上的开放商业层

路由服务对 Agent 提供一个简单接口（可做成 OpenAI 兼容格式），背后完成"查挂单 → 选服务商 → 发请求 → 失败重试"的循环。

- **非托管**：路由服务只编排信息，不经手资金，通道仍由调用方直连服务商；
- **可竞争**：任何人都可以运营路由服务，优势来自更新鲜的健康度数据与更好的选择策略；
- **商业收入与 VSR 隔离**：路由服务的收入是链下商业协议，不进入收据结算。

愿意自己动手的高级 Agent 可以绕过路由服务，直接与协议交互。

多方分账：以 AgentStore 为例

Agent A 接到用户任务，调用 Agent B，B 再调用模型服务商 C。三次调用各自生成 VSR 收据，通过 `rootCallId` 串成一条调用链。部署在合约层的分账合约读取这条链，按 A、B 预先设定的规则自动分配收入。

分账规则是合约，不是链内核：Agent 商业模式变化快，社区可以随时发布新的分账标准，而不需要修改协议。